VEDANT GUPTA

+91 9098523827 | vedantgupta_23mc171@dtu.ac.in | Delhi, India
LinkedIn | GitHub | LeetCode

Education

Delhi Technological University (DTU) — New Delhi, India
B.Tech in Mathematics & Computing | CGPA: 9.082 — *2023 – 2027*

Experience

Center for Railway Information Systems (CRIS) — Delhi, India
AI Full Stack Developer Intern — *June 2026 – July 2026*

- Eliminated monitoring agent deployment on remote cloud servers by architecting an asynchronous SSH2 polling service in `sshService.js`, removing all local CPU/RAM overhead while executing Linux commands and parsing stdout via custom regex.
- Accelerated infrastructure troubleshooting by reducing average diagnostic time from minutes to **<800ms** through a React AI chatbot interface integrated with an **LLM orchestration layer** translating natural language into structured database queries over historical server metrics.
- Automated manual compliance auditing, cutting review time by **95%** (**8+ hours/month**) and eliminating server registration errors via a transient connection-validation React form with automated Excel/HTML report generation.

Projects

Botlock — Decentralized AI Agent Paywall & Micropayment System — GitHub | Live

- **Tech Stack:** Node.js, Express.js, Solana Web3.js, Supabase (PostgreSQL), Next.js, Docker, Railway, Vercel, npm
- **Built a protocol-level content gating gateway** returning HTTP 402 with HMAC-SHA256 signed challenges to unauthorized AI crawlers (GPTBot, ClaudeBot, PerplexityBot). On-chain USDC verification via Solana RPC pre/post token balance delta completes in ∼400ms at <$0.001 per transaction, with **O(1)** PostgreSQL replay cache achieving **100% double-spend mitigation**. Mock verification mode cuts local dev latency by 97.5% (**∼400ms to <10ms**).
- **Published two npm packages** — **botlock-sdk** (zero-dependency, framework-agnostic core, 33 bot-detection patterns, **0ms human-passthrough overhead**, 4 framework adapters) and **botlock-agent-sdk** (drop-in fetch() replacement, automatic 402 resolution, 6 typed error codes, per-request & lifetime budget caps, concurrent payment de-duplication via in-flight promise coalescing) — reducing publisher integration to a **2-line middleware** and agent integration to a single import swap.
- Developed a multi-signal bot detection pipeline with **O(1)** bitwise datacenter CIDR matching, header fingerprint scoring, and a 2,000-entry LRU-cached reverse DNS queue, achieving **<0.1% false-positive rate** for human traffic. Designed a SIWS-authenticated publisher dashboard with stateless HMAC-signed sessions (**0ms validation**, no database reads), 6-metric analytics grid, and 15-second auto-refresh.

NexusAI — Distributed Multi-Agent AI Platform with Microservices Orchestration — GitHub | Live

- **Tech Stack:** Node.js, Express 5, React 19, LangChain, LangGraph, Redis, MongoDB, AWS S3, Firebase, Razorpay, Docker
- Architected **4 independent microservices** (Auth, Chat, Agent, Billing) behind a centralized API Gateway with HttpOnly Redis-backed session cookies (7-day TTL). Gateway strips raw credentials before proxying and injects trusted x-user-* headers, eliminating lateral credential propagation across all downstream services. Billing service handles full Razorpay lifecycle with HMAC-SHA256 verification and atomic cross-service plan sync, achieving **zero stale-state reads** post-upgrade.
- Constructed an **8-agent LangGraph StateGraph orchestration engine** with a two-phase supervisor router — deterministic MIME-type fast-path for file uploads followed by LLM-based semantic classification — achieving **sub-200ms intent routing** on ambiguous prompts. Hardened the system with per-user, per-agent **Redis atomic rate limiting (3–20 req/min)** and tiered credit deduction (1–10 credits per request) with real-time balance propagation, and a 20-message sliding-window conversation memory with 24h TTL.
- Devised an ephemeral PDF RAG pipeline performing RecursiveCharacterTextSplitter chunking **(1000 chars, 200 overlap)**, gemini-embedding-001 vectorization, and top-5 Qdrant similarity search — with automatic resource cleanup in a **finally** block. Delivered a React 19 frontend with Monaco Editor, sandboxed iframe live code preview (**0ms server round-trip), one-click S3 deployment**, and Razorpay billing drawer with 4-tier credit plans (Free/Starter/Pro).

FETCHIT — Cross-Platform Open Source Command Line Interface Video Downloader — GitHub | npm

- **Tech Stack:** TypeScript, React 19, Ink, Node.js, yt-dlp, ffmpeg, npm
- Built **fetchit, an open-source CLI video downloader** supporting **2,000+ sites** across Windows, macOS, and Linux, **published to npm as @vedant1521/fetchit.**
- Integrated playlist auto-detection and batch download with a multi-select UI, downloading selected items into numbered files under ∼/Downloads/<playlist>/ with per-item progress (e.g., item 3/12).
- Engineered honest download-size labels by pinning exact yt-dlp format_ids to probed metadata, **fixing a bug** where an unconstrained fallback selector let a **144p label deliver a 232 MB 4K file** — a **46× size discrepancy**.

Technical Skills

Languages: C/C++, JavaScript, TypeScript, Python, SQL, HTML, CSS
Frameworks: React.js, Node.js, Express.js, Next.js, LangChain, LangGraph, Framer Motion, Tailwind CSS
Databases & Cloud: PostgreSQL, MongoDB, Redis, AWS S3, Supabase | **DevOps**: Docker, Railway, Vercel, GitHub Actions
Tools: Postman, npm, LaTeX | **Coursework**: OOP, Computer Networks, OS, DBMS, DSA

Academic Achievements and Awards

- Achieved a perfect **10.0 SGPA** and secured **University Rank 1** in the 5th semester.
- Ranked in the top **1.9%** of over **1.1 million** candidates in JEE Mains 2023 and qualified the prestigious **NTSE Stage-1** in 2021.